Weekly Rapport — Weeks 2–3

Real-Time Anomaly Detection Platform for Branch Teller Operations

Yassine — Amen Bank, Direction Centrale du Système d'Information

June 8–17, 2026

Contents

1 Executive Summary

This rapport covers two weeks of intensive development (June 8–17), during which the project transitioned from the source extraction and design phase into full implementation. The period produced six major deliverables: completion of the CTAF source extraction with architectural decisions, design and calibration of five client archetypes, a fully operational synthetic data generator producing 340,839 labeled transactions, a PostgreSQL cold store with loaded data, a batch profile computation pipeline producing 260,000 weekly client profiles and 7,800 weekly employee profiles, and a batch enrichment script producing 340,839 enriched training events with 74 features. The project is now ready for model training.

2 Source Extraction Completion (June 8–10)

2.1 CTAF 2021 Annual Report — Case Studies

Four typological case studies were extracted from the CTAF (Commission Tunisienne des Analyses Financières) 2021 Annual Report using a structured 7-field template. Each case surfaced schema amendments across three layers (raw event, client profile, enriched event):

- **Case 1 (Espèces-heavy structuring):** Multiple versements just below the 10,000 DT reporting threshold, dispersed across branches. Informed the near-threshold ratio feature and branch distribution drift detection.
- **Case 2 (Virement chain):** Series of virements to unknown beneficiaries with round amounts. Informed the first-time beneficiary detection and round-amount flagging.
- **Case 3 (PPE exploitation):** Politically exposed person used as a conduit. Validated the PPE binary flag layered on archetypes.
- **Case 4 (Remise de chèques laundering):** Third-party cheques deposited in rapid succession. Informed cumulative amount tracking and cheque-specific anomaly patterns.

2.2 Structural Extractions

Six quantitative extraction targets were completed:

- Instrument co-occurrence rates: espèces 68%, virement 56%, chèque 34%
- BBE (Billets de Banque Étrangers) concentration: 85% of operations in the 5,000–20,000 DT band
- Predicate offence taxonomy: contrebande 20%, corruption 19%, escroquerie 11%
- Geographic concentration across Tunisian governorates
- Two directly encodable regulatory rules

2.3 Architectural Decisions D19–D22

Four new decisions were locked based on CTAF evidence:

- **D19:** Evidence-gated sector risk scoring — sector risk weights backed by CTAF data, non-evidenced sectors scored at zero
- **D20:** `regulatory_client_type` (occasional/habitual) and `maturity_status` (new/mature) as independent fields, not conflated
- **D21:** Multi-tier anomaly difficulty gradient (easy/medium/hard) for controlled injection
- **D22:** Multi-operation anomaly sequences as the injection format for hard scenarios

2.4 Source Phase Closure

BCT (Banque Centrale de Tunisie) circulars 2018-09, 2017-08, and 2021-05 were confirmed as restricted documents not publicly available. The source extraction phase was closed with three sources: S1 (PaySim), S2 (Amen Bank 2023 Annual Report), and S3 (CTAF 2021). BCT circulars are documented as a known gap. An 18-bucket sector taxonomy was enumerated for the `accounts.sector_code` field.

3 Archetype Design and Calibration (June 12)

3.1 Five Client Archetypes

Five archetypes were designed across six behavioral dimensions (operation mix, frequency, timing, amount ranges, counterparty patterns, branch loyalty), calibrated using Amen Bank's deposit composition data:

Archetype	Population	Dominant Operation	Monthly Volume	Amount Range (DT)
Salaried	49%	Retrait (50%)	~5.7 ops	200–500
Small Business	33%	Versement (55%)	~37.5 ops	500–2,000
Student	5%	Retrait (95%)	~7.2 ops	40–80
Retiree	7%	Retrait/Virement	~3.2 ops	200–500
Big Business	6%	Virement (78%)	~63 ops	2,000–20,000

Population proportions derived from Amen Bank's 61% individual / 33% private business / 6% institutional split, with the individual bucket subdivided ~80% salaried / 12% retiree / 8% student.

3.2 Law n°41-2024 Impact

A regulatory discovery during archetype calibration: Law n°41-2024 (effective February 2025) caps individual cheques at 30,000 DT, confirmed by BCT Q1 2026 data showing a 24.9% volume collapse in cheque usage. This revised the big business archetype: remise de chèques dropped from ~12–15/month to ~2–4/month, with virement absorbing the difference. A new anomaly scenario (D23) was added: cheque structuring just under the 30,000 DT ceiling.

3.3 LSTM Scope Correction

The project scope is "opérations de caisse en agence bancaire" — all four operations are teller window (guichet) transactions. Channel is constant, so it was dropped as an LSTM prediction head, reducing from 7 to 6 multi-task heads.

4 Synthetic Data Generator (June 12)

4.1 Architecture

The generator follows a 4-class, 2-phase design:

- **Archetype** (Layer 1): Loads behavioral parameters from `config/archetypes.yaml` via pyyaml
- **Client** (Layer 2): Gaussian personality sampling with `np.clip` constraints, operation mix normalization, progressive std narrowing
- **Generator** (orchestrator): Two-phase execution — planning phase assigns anomaly scenarios and time windows, generation phase produces events
- **Event** (inline): Per-event noise and payload construction

4.2 Validation Results

At 10,000 clients over 180 simulated days (January 1 – June 29, 2025):

- **Total events:** 340,839
- **Anomaly rate:** 0.147% (target: 0.13%, acceptable)
- **All 13 anomaly scenario types represented**
- **Amount calibration:** Salaried retrait mean ~300 DT (consistent with Tunisian salary ~1,350 DT/month)

A critical bug was identified and fixed: the anomaly planning logic originally inflated the anomaly rate from 0.13% to 1.52% because multi-day plan entries produced many events rather than one. The fix divides target events by expected events per entry.

4.3 Output Format

Single chronologically ordered CSV with envelope columns (event_id, client_id, account_id, employee_id, branch_id, timestamp, amount, currency, channel, operation_type) plus a JSON payload column, labels (is_anomaly, anomaly_type). Loaded into PostgreSQL via COPY protocol.

5 Data Infrastructure (June 16)

5.1 Generator Extensions

The generator was extended to produce four CSV files per run:

File	Rows	Purpose
transactions.csv	340,839	All simulated events
clients_master.csv	10,000	Client reference data
accounts.csv	10,000	Account reference data
employees_master.csv	300	Employee reference data (2 per branch)

Key additions: branch-linked employee assignment (replacing random assignment), `account_opening_date` per client (archetype-calibrated ranges), `client_type` regulatory classification (personne_physique vs personne_morale), `maturity_status` cold-start flag.

5.2 PostgreSQL Cold Store

PostgreSQL 17 stood up locally with database `amen_anomaly`. Five tables created with foreign-key-respecting load order:

1. `clients_master` (PK: client_id)
2. `accounts` (FK → clients_master)
3. `employees_master` (PK: employee_id)
4. `transactions` (FK → clients_master, employees_master)
5. `profile_snapshots` (FK → clients_master)

Data loaded via psycopg2's `copy_expert` using PostgreSQL's COPY protocol. Indexes created on `(client_id, timestamp)` and `(employee_id, timestamp)` for efficient profile computation.

5.3 Batch Profile Computation

The batch profile script (`src/batch/batch_profile.py`) computes 122-field client profiles stored as JSONB in `profile_snapshots`:

- **Category 1 — Personal info** (5 fields): archetype, client_type, home_branch_id, account_age_days, maturity_status
- **Category 2 — Transaction stats** (96 fields): 4 operations × 4 windows (24h/7d/30d/90d) × 6 statistics (count, sum, mean, std, min, max)
- **Category 3 — Behavioral patterns** (14 fields): 5 distributions at 30d/180d (operation, hour, day-of-week, branch, amount buckets) plus known counterparty sets
- **Category 4 — Financial state** (6 fields): inflow/outflow/net at 30d/90d windows
- **Category 6 — LSTM buffer** (1 field): Last 50 events ordered chronologically

6 Training Data Pipeline (June 17)

6.1 Point-in-Time Profile Design

A key design decision: training data must use **point-in-time profiles** to avoid data leakage. Using the final simulation profile (June 29) to enrich a January transaction would leak future information. The solution: compute weekly profile snapshots across the simulation period.

Computation budget: The batch profile script takes ~15 seconds per snapshot. 26 weekly snapshots × 15 seconds 6.5 minutes — well within acceptable range. Weekly granularity was chosen to capture medium-difficulty anomaly patterns (5-day duration) while remaining computationally cheap.

6.1.1 Weekly Profile Snapshots Produced

Asset	Count	Method
Client profiles	260,000 (26 weeks × 10,000 clients)	`batch_profile.py` with weekly date loop
Employee profiles	7,800 (26 weeks × 300 employees)	`batch_employee_profile.py`
Archetype baselines	5 (one per archetype)	`batch_archetype_baselines.py`

Profile matching uses `pd.merge_asof` — an as-of join that, for each transaction, selects the most recent profile snapshot *before* the transaction timestamp, preventing future data leakage.

6.2 Employee Profile Design

Employee profiles follow the same batch pattern as client profiles, with key differences:

- **Client distribution** added to behavioral patterns — tracks which clients each employee serves, detecting concentration patterns indicating potential collusion
- **Peer comparison** (new category): 5 z-scores at 30d/90d windows comparing each employee against bank-wide peers

6.2.1 Peer Comparison Normalization Strategy

A design challenge: with only 2 tellers per branch, per-branch z-scores are statistically meaningless. The solution uses a **split normalization approach**:

- **Volume metric:** Normalized per-branch first (employee_volume / branch_average_volume), then z-scored bank-wide. This prevents high-volume branches from appearing as outliers while still detecting employees who handle disproportionate volume *within* their branch.
- **Ratio metrics** (near-threshold ratio, error rate, client concentration, client-location mismatch): Z-scored directly bank-wide, since ratios are already volume-independent.

This design reflects real-world Tunisian banking context: branch workloads vary significantly (e.g., Tunis center vs. suburban branches), making raw volume comparison across branches misleading.

6.3 Balance Computation

The enriched event schema requires `current_balance` per transaction. Since the generator doesn't produce balance data, balance is derived from:

$$\text{balance}(t) = \text{initial\_balance} + \sum_{i<t} \text{signed\_amount}_i$$

where versements and cheques are positive (inflow) and retraits and virements are negative (outflow). Initial balances were added to `clients_master` via ALTER TABLE, sampled per archetype:

Archetype	Balance Range (DT)
Salaried	1,000 – 3,000
Small Business	5,000 – 20,000
Student	100 – 1,000
Retiree	2,000 – 8,000
Big Business	50,000 – 200,000

6.4 Batch Enrichment

The batch enrichment script (`src/batch/batch_enrichment.py`) produces the complete training dataset by joining each transaction with its point-in-time profile and computing all derived features:

Output: 340,839 enriched events × 74 columns → `data/enriched_events.csv`

The 74 columns span 11 sections of the enriched event schema:

1. **Metadata** — event IDs, timestamps, profile snapshot references
2. **Raw event passthrough** — operation type, amount, branch, employee
3. **Cold-start flags** — is_client_mature, is_employee_established, archetype
4. **Regulatory signals** — threshold flags (10,000 DT), near-threshold band (8,000–9,999 DT), business hours/days, duplicate detection
5. **Client-side features** — z-scores (amount vs 30d/90d profile), frequency lookups (branch, hour, day-of-week from profile distributions), counterparty novelty (first-time beneficiary/emitter), cumulative amounts (24h/7d), balance features
6. **Archetype baselines** — z-scores against archetype average (cold-start fallback)
7. **Behavioral drift** — Jensen-Shannon divergence between 30d and 180d distributions
8. **Employee-side features** — z-scores from employee profile, client-location mismatch
9. **Peer comparison** — passthrough of 5 peer z-scores at 30d/90d
10. **Risk history** — all zeros (no alerts yet — realistic initial state)
11. **LSTM buffer** — recent 50 events from profile

6.4.1 Jensen-Shannon Divergence for Behavioral Drift

A key feature engineering technique: JS-divergence measures how much a client's recent behavior (30-day window) has shifted from their long-term pattern (180-day window). Unlike comparing individual statistics, JS-divergence captures the **overall distribution shape change** in a single scalar value.

For example, if a client's operation mix shifts from {retrait: 50%, virement: 40%, versement: 10%} to {retrait: 80%, virement: 15%, versement: 5%}, JS-divergence produces one number quantifying this shift. A value near 0 indicates stable behavior; a high value signals behavioral change — exactly the pattern produced by hard-tier anomaly scenarios like gradual behavioral drift.

Five drift features are computed: branch, hour, day-of-week, operation mix, and employee (client distribution from the employee's perspective).

6.4.2 Validation and Known Issues

Validation revealed that 14,201 transactions (before the first weekly snapshot at January 8) have no profile match, producing distorted z-scores (e.g., z = amount / 1 instead of a meaningful standardized score). These will be filtered during training data preparation using a 3-week warm-up period, leaving ~305,000+ usable training events.

7 Architecture Discussion: Scaling and Language Choice

A design discussion addressed whether Python is suitable for the streaming pipeline given latency constraints. Analysis of the per-event latency budget:

Component	Estimated Latency
Kafka consume	~5 ms
Redis lookup (2 profiles)	~2 ms
Enrichment computation	~1–2 ms
Model inference (PyTorch)	~10–20 ms

Component	Estimated Latency
Decision logic	~1 ms
Kafka produce	~15 ms
Total	**~35–45 ms**

The 200 ms target for near-real-time branch supervisor alerts is comfortably met with Python. The argument for JVM-based streaming (Scala/Flink) applies at throughput levels of millions of events per second — far beyond the ~hundreds of events per minute from 150 branches.

The architecture is **language-agnostic at service boundaries** (Kafka topics are the contract), enabling a migration path to JVM-based streaming if throughput requirements grow. This is documented as a production roadmap item rather than a current requirement.

8 Project Structure

```
src/
  generator/
    __init__.py
    archetype.py      # Layer 1: YAML-driven archetypes
    client.py          # Layer 2: Gaussian personality sampling
    generator.py       # Orchestrator: planning + generation
    main.py            # Entry point
  batch/
    __init__.py
    batch_profile.py             # Weekly client profiles (122 fields)
    batch_employee_profile.py    # Weekly employee profiles + peer comparison
    batch_archetype_baselines.py # Archetype aggregate baselines
    batch_enrichment.py          # Training data enrichment (74 features)
  services/
  dashboard/
config/
  archetypes.yaml    # Externalized archetype parameters
data/
  transactions.csv        # 340,839 raw events
  clients_master.csv      # 10,000 clients
  accounts.csv            # 10,000 accounts
  employees_master.csv    # 300 employees
  enriched_events.csv     # 340,839 × 74 training data
models/
docker/
tests/
```

9 Summary of Deliverables

Deliverable	Status	Key Metric
CTAF extraction + S3 checkpoint	Complete	4 cases, 6 structural targets, D19–D22
Archetype design + YAML config	Complete	5 archetypes, 6 dimensions each
Synthetic data generator	Complete	340,839 events, 0.147% anomaly rate
PostgreSQL cold store	Complete	5 tables, COPY-loaded
Weekly client profiles	Complete	260,000 snapshots (26 × 10,000)
Weekly employee profiles	Complete	7,800 snapshots (26 × 300)
Archetype baselines	Complete	5 baselines
Batch enrichment (training data)	Complete	340,839 × 74 features

10 Next Steps

1. **Model training preparation:** Filter enriched events (warm-up period), split into Autoencoder dataset (normal events only) and LSTM dataset (sliding window sequences of 50)
2. **Autoencoder implementation:** PyTorch architecture, reconstruction-error-based anomaly scoring, threshold calibration
3. **LSTM implementation:** 6 multi-task prediction heads, sequence input from enriched events
4. **MLflow integration:** Experiment tracking, model versioning
5. **Score fusion strategy:** Combine Autoencoder and LSTM scores for the final anomaly decision